

Self-Reflective Report

BIT1123 Object Oriented Programming (Java) — Assignment 1 (Individual, 20%)

Item	Detail
Student Name	Ali Sharif AbdulKadir Sharif
Student ID	202505010493
Program	BCSSE — Bachelor of Computer Science (Software Engineering)
Class Code	BIT1123
Course	BIT1123 Object Oriented Programming (Java)
Lecturer	Sir Nazmirul Izzad Bin Nassir
Faculty	Faculty of Information Technology, City University Malaysia — Cyberjaya Campus

1. Introduction

This report reflects on my learning journey through the ten practical tutorials of BIT1123 Object Oriented Programming. Over the semester the work progressed from a single `System.out.println` statement in Week 1 to a complete event-driven Swing desktop application in Week 10, and consolidating all of it into one repository gave me an unusually clear view of that progression.

Before this course I thought of a program as a list of instructions to be executed top to bottom. What changed is that I now think first about *which objects exist in the problem*, what each one is responsible for, and how they should communicate — and only then about the statements that implement it. This report records what I learned in each tutorial, the specific problems I ran into, how I resolved them, and where I intend to take these skills next.

2. Knowledge Gained from Each Tutorial

Week 1 — Java Fundamentals. The mechanics of the language: primitive types, operators, conditionals, loops and arrays. Writing `StudentGrade.java` taught me more than the syntax alone, because extracting the grade logic into a reusable `calculateGrade()` method instead of repeating an `if / else` chain inside the loop was my first real experience of factoring out a responsibility.

Week 2 — Classes and Objects. The shift from data to *modelled* data. Defining `Student` with attributes, a constructor and behaviour, then creating two instances in `Mean.java`, made the distinction

between a class and an object concrete: one blueprint, many independent objects each holding their own state.

Week 3–4 — Inheritance and Polymorphism. The pivotal tutorial. `Person` was extended by `Student` and `Lecturer`, each overriding `introduce()`. Declaring all three as `Person` references and calling the same method on each — and getting three different outputs — is what made runtime polymorphism click for me.

Week 5 — Encapsulation. Making every field `private` and routing all access through getters and setters. The lesson was that a class's public surface is a deliberate design decision, not an afterthought.

Week 6 — Protected Access and Inheritance. `Employee` exposed `protected` fields so `Lecturer` could use them directly while they stayed hidden from unrelated classes. This filled in the middle ground between `public` and `private` that Week 5 had left open.

Week 7 — Abstraction. `Appliance` as an abstract class with shared concrete behaviour plus an abstract `operate()` that each subclass had to implement. Seeing `WashingMachine`, `Refrigerator` and `AirConditioner` all driven through a single `Appliance` reference showed me the value of programming to a supertype rather than to concrete classes.

Week 8–9 — Collections, File I/O and Exceptions. `ArrayList` for storage that grows as needed, `BufferedWriter` and `BufferedReader` for persistence, and try-with-resources so streams close even when something fails. This tutorial introduced the idea that failure is a normal state a program must be designed for, not an exceptional one to be ignored.

Week 10 — Swing GUI Mini Project. Building *Code Boss Battle* required composing nested panels, wiring `ActionListener` handlers through lambdas, animating with `javax.swing.Timer`, and keeping game state consistent across every interaction. Crucially, it required separating the model (`Questions`) from the view (`QuizBattleGUI`) — the first time I designed a program around that boundary rather than putting everything into one file.

3. Challenges Encountered

Understanding polymorphism. In Week 3–4 I could not initially see why `Person p2 = new Student(...)` was useful. If the variable is declared as `Person`, why not simply declare it as `Student`? The purpose of the indirection was not obvious to me.

Package declarations that did not match their folders. Several tutorials declared packages such as `package week_3_4;` inside a folder named `week_3-4`. The files compiled but would not run with a plain `java Main`, and the error messages about the class not being found did not point at the real cause.

Deciding what belongs in a class. In Week 6 I was unsure whether `subject` and `department` belonged on `Employee` or on `Lecturer`. Both placements compiled and ran, so the compiler gave me no signal about which was the better design.

Abstract classes felt like extra ceremony. In Week 7 my instinct was that if `operate()` has no body, it should simply be left out. Writing a method signature with no implementation seemed like wasted effort.

Swing's complexity in Week 10. The GUI was by far the hardest tutorial. Layout managers did not place components where I expected, and my first version put question data, game rules and interface code together in one class — which became very difficult to change.

Code that only worked on my machine. Two problems surfaced only when the project was run elsewhere. `QuizBattleScreenshot.java` contained an absolute path (`/workspaces/...`) that existed on no other computer, and the boss character was rendered with `Segoe UI Emoji`, a Windows font, so on other systems it displayed as an empty box. Both worked perfectly where I wrote them, which is exactly why I did not notice.

Weak version control habits. Reviewing the repository history, most of my commit messages read simply `done`. When I needed to find where a specific change had been introduced, the history told me almost nothing.

4. How the Challenges Were Overcome

Polymorphism became clear through deliberate experiment rather than re-reading notes. I changed `p2.introduce()` to call the `Student` version, then the `Lecturer` version, and watched the output change while the declaring type stayed `Person`. Seeing that Java resolves the call against the actual object at runtime — not the declared reference type — is what made the concept concrete.

The package mismatches I resolved by removing the package declarations entirely, so each week folder is self-contained in the default package. Every folder now compiles and runs with an identical pair of commands, which also made the "How to Run" section of the README straightforward to write. The wider lesson was that the compiler accepting my code is not the same as my code being correct.

For the class design question in Week 6, I applied a simple test: does every `Employee` have a subject? A receptionist does not, so `subject` belongs on `Lecturer`. Asking whether an attribute is true of *every* instance of the base class became my default check.

Abstract classes made sense once I deliberately broke one. I removed `operate()` from a subclass and the compiler refused to build — which was the point. The abstract method is a contract enforced at compile time, and that guarantee is what lets `Main` call `operate()` on any `Appliance` with confidence.

Swing I approached by breaking the interface into separate builder methods — `buildTopPanel()`, `buildBossPanel()`, `buildBottomPanel()` — instead of assembling everything in the constructor. I then moved all question data into its own `Questions` class with a static factory method. Once the model was separated from the view, changing the question bank no longer meant touching interface code at all.

The machine-specific problems I fixed by removing the hardcoded path in favour of a relative one supplied as an argument, and by replacing the emoji with a monospaced text drawing that renders

identically on every platform. I also added a headless check so the program falls back to a console simulation when no display is available. Testing on a different operating system, I learned, finds an entire class of bug that testing on your own machine never will.

On version control, I now write commit messages that describe the change — what it does and why — rather than recording that work happened.

5. Improvements in Java Programming Skills

The clearest technical improvements are these. I can compile and run multi-class Java programs from the command line and read compiler errors as useful diagnostics rather than obstacles. I use `private` fields with accessors by default instead of exposing state directly. I can build a class hierarchy that uses inheritance because the relationship genuinely is "is-a", not merely to avoid retyping code.

Beyond the language itself, three habits changed. I now split responsibilities across separate files rather than growing one large class — Week 7 has five focused files and is far easier to follow than a single combined one would be. I write code that anticipates failure, using try-with-resources and handling `IOException` explicitly. And I check that a program works outside my own environment, which is a discipline the Week 10 font and path bugs taught me directly.

My debugging approach also matured. Early in the semester I would change lines until the error disappeared. Now I read the error, form a hypothesis about the cause, and test that hypothesis — a much faster process, and one that leaves me actually understanding the fix.

6. Understanding of Object-Oriented Programming Concepts

Encapsulation is the practice of keeping a class's internal state private and exposing a deliberate public interface. Its value is that the internals can change freely without breaking any calling code, because nothing outside the class depends on them. Week 5's `Student` demonstrates this: the fields are entirely inaccessible from outside, and all interaction passes through methods the class controls.

Inheritance lets a subclass acquire the fields and behaviour of a superclass and extend them, expressing an "is-a" relationship — a `Lecturer` *is an* `Employee`. Its real benefit is not reduced typing but a shared type that lets different objects be handled uniformly. I also learned its limits: inheritance is the wrong tool when the relationship is "has-a", where composition fits better.

Polymorphism allows one interface to serve many implementations, so the same call produces the appropriate behaviour for whatever object is actually present. Week 3–4 shows this most directly: three `Person` references, one method call, three different outputs, with the decision made at runtime. Week 7 applies the same principle usefully — `Main` calls `operate()` without knowing or caring which appliance it holds.

Abstraction means exposing what a type does while hiding how it does it. `Appliance` declares that every appliance can `operate()`, and each subclass decides what that means. As a design tool, abstraction is what lets a program depend on a stable contract rather than on volatile implementation details.

Consolidating the repository showed me these four principles are not independent techniques but one connected approach to structuring a system — encapsulation defines the boundaries, inheritance and abstraction define the relationships across those boundaries, and polymorphism is what makes those relationships useful at runtime.

7. Future Learning Plans

In the short term I want to strengthen the fundamentals this course opened up: interfaces and multiple inheritance of type, generics beyond the basic `ArrayList<String>` usage in Week 8–9, and the wider Collections Framework — `HashMap`, `Set` and the differences between implementations.

Next I intend to learn unit testing with JUnit. Every tutorial in this repository was verified by running it and reading the output, which does not scale and does not protect against regressions. Automated tests would have caught the Week 10 path bug immediately.

Beyond that, my goals are to study common design patterns — Factory, Observer, Strategy — since `Questions.loadQuestions()` is effectively a static factory I arrived at by intuition rather than by knowing the pattern; to deepen my Git practice with branching and pull requests instead of committing directly to `main`; and to build a larger project that connects a Java application to a database, so that data persists in something more capable than a text file.

Longer term I want to explore Java for backend development with Spring Boot, and to apply object-oriented design in another language to see which principles are universal and which are specific to Java.

8. Conclusion

The most valuable outcome of this course is not any single language feature but a change in how I approach a problem. I now start by asking what objects the problem contains and what each should be responsible for, and I have found that decisions made at that level determine how difficult everything afterwards becomes.

Consolidating ten weeks of tutorials into one repository was itself instructive. Reviewing the earlier work with what I know now, I could see clearly where the design was weak — the empty Week 1 files, the mismatched package declarations, the machine-specific paths — and I was able to fix them. Being able to critique my own earlier code is, I think, the clearest evidence that the understanding is real.

The Week 10 project brought everything together. *Code Boss Battle* required encapsulation for the question model, composition for the interface, polymorphism through event handlers, exception handling for file operations, and a deliberate separation between model and view. None of that was possible in Week 1, and being able to build it is a concrete measure of what the semester produced.

9. GitHub Repository URL

<https://github.com/SHAFOO11/ALI-SHARIF-ABDULKADIR-SHARIF--202505010493-JAVA-PROGRAMMING>

Submitted for BIT1123 Object Oriented Programming (Java), Assignment 1 — Faculty of Information Technology, City University Malaysia, Cyberjaya Campus.